DEBRE MARKOS UNIVERSITY

INSTITUTE OF TECHNOLOGY

SCHOOL OF COMPUTING

SOFTWARE ENGINEERING DEPARTMENT

Project Title: ETHIOGEBIYA - ONLINE MARKETPLACE AND BROKERAGE PLATFORM

Prepared By:

Name	ID No.	Signature
Alemu Eyayu	153794	_____
Abrham Belsty	153732	_____
Abrham Yeshtie	153736	_____
Hawltu Mehari	154166	_____

Project Advisor:

Submission Date:

ACKNOWLEDGEMENT

First and foremost, we would like to thank GOD for His grace and guidance throughout this project work.

We extend our sincere gratitude to our advisor for his valuable guidance, comments, and suggestions that helped us shape this project.

We would also like to thank the traditional brokers, buyers, sellers, and company representatives across Ethiopia who participated in interviews and provided essential information about the current brokerage system.

Finally, we thank our families and friends for their encouragement and support during this project.

TABLE OF CONTENTS

Section	Title	Page
	CHAPTER ONE: INTRODUCTION	
1.1	Background of the Project	4
1.2	Statement of the Problem	5
1.3	Literature Review	6

Section	Title	Page
1.4	Objective of the Project	7
1.4.1	General Objective	7
1.4.2	Specific Objectives	7
1.5	Scope of the Project	8
1.6	Significance of the Project	9
1.7	Tools and Methodology	9
1.7.1	Data Collection Methodology	9
1.7.2	Programming Languages to be Used	10
1.7.3	System Requirements (Hardware and Software)	11
1.7.4	System Modeling Tools	11
1.8	Feasibility Study	12
	CHAPTER TWO: SYSTEM ANALYSIS	
2.1	Overview of the Existing System	14
2.2	System Requirement Specification	15
2.3	Functional Requirements	15
2.4	Non-Functional Requirements	17
2.5	Overview of the Proposed System	19
2.6	Business Rule Identification	21
2.7	System Requirement Analysis	22
2.8	Actors and Use Case Identification	23
2.9	Use Case Diagram	24
2.10	Use Case Description	25
2.11	Sequence Diagram	31
2.12	Activity Diagram	34
2.13	Analysis Class Diagram	37
	CHAPTER THREE: SYSTEM DESIGN	
3.1	Design Class Diagram	38
3.2	Physical Data Model	38
3.3	User Interface Design	44
3.4	Deployment Diagram	47

Section	Title	Page
	References	47

LIST OF TABLES

Table No.	Description	Page
Table 1	Actors of the System	23
Table 2	Use Case Description for Login	25
Table 3	Use Case Description for User Registration	26
Table 4	Use Case Description for Verify Account	27
Table 5	Use Case Description for Search Products	27
Table 6	Use Case Description for Send Message	28
Table 7	Use Case Description for Manage User	29
Table 8	Use Case Description for Process Payment	29
Table 9	Use Case Description for View Reports	30
Table 10	Users Collection Structure	41
Table 11	Listings Collection Structure	41
Table 12	Companies Collection Structure	42
Table 13	Categories Collection Structure	42
Table 14	Conversations Collection Structure	42
Table 15	Messages Collection Structure	42
Table 16	PremiumSubscriptions Collection Structure	43
Table 17	Notifications Collection Structure	43
Table 18	SavedSearches Collection Structure	43
Table 19	CompanyMembers Collection Structure	44
Table 20	ActivityLogs Collection Structure	44
Table 21	ModerationReports Collection Structure	44

LIST OF FIGURES

Figure No.	Description	Page
Figure 1	Use Case Diagram	25
Figure 2	Sequence Diagram for User Registration	31

Figure No.	Description	Page
Figure 3	Sequence Diagram for Post Listing	31
Figure 4	Sequence Diagram for Search Products	32
Figure 5	Sequence Diagram for Message Exchange	33
Figure 6	Sequence Diagram for Process Payment	33
Figure 8	Activity Diagram for Login	34
Figure 9	Activity Diagram for User Registration	34
Figure 10	Activity Diagram for Post Listing	35
Figure 11	Activity Diagram for Search Products	36
Figure 12	Activity Diagram for Premium Subscription	37
Figure 13	Activity Diagram for Message Exchange	37
Figure 14	Analysis Class Diagram	37
Figure 15	Design Class Diagram	38
Figure 16	Physical Data Model	38
Figure 17	Login Interface Design	44
Figure 18	User Dashboard Interface	45
Figure 19	Product Listing Interface	46
Figure 20	Deployment Diagram	47

ABBREVIATIONS AND ACRONYMS

Abbreviation	Meaning
API	Application Programming Interface
CSS	Cascading Style Sheets
DBMS	Database Management System
ERD	Entity Relationship Diagram
Express	Express.js Web Framework
GPS	Global Positioning System
GUI	Graphical User Interface
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
ID	Identification

Abbreviation	Meaning
JSON	JavaScript Object Notation
JWT	JSON Web Token
KYC	Know Your Customer
MERN	MongoDB, Express, React, Node.js
MongoDB	NoSQL Database
MVC	Model-View-Controller
MVP	Minimum Viable Product
Node.js	JavaScript Runtime Environment
OOP	Object-Oriented Programming
OTP	One-Time Password
PWA	Progressive Web Application
React	React.js Frontend Library
Redux	Predictable State Container
REST	Representational State Transfer
SMS	Short Message Service
SSL	Secure Sockets Layer
UI	User Interface
UML	Unified Modeling Language
USSD	Unstructured Supplementary Service Data
UX	User Experience
Vite	Frontend Build Tool

CHAPTER ONE: INTRODUCTION

1.1 Background of the Project

In Ethiopia, the brokerage system (known locally as "Delala") has been an integral part of commerce for generations. Traditional brokers serve as intermediaries connecting buyers and sellers across various sectors including real estate, automotive, agricultural products, and general merchandise. These brokers facilitate transactions by leveraging their personal networks, market knowledge, and negotiation skills. However, this system operates largely through informal channels, face-to-face interactions, and word-of-mouth referrals.

The Ethiopian economy has been experiencing significant growth and digital transformation in recent years. With the government's commitment to the Digital Ethiopia 2025 strategy, there is an increasing push toward

digital solutions that can address traditional market inefficiencies. The brokerage sector, which handles billions of Birr in transactions annually, remains largely untouched by this digital revolution.

Debre Markos University, like many institutions in Ethiopia, recognizes the need for digital transformation in commercial activities. The traditional brokerage model presents several challenges in the modern context: limited geographical reach across Ethiopia's vast landscape, information asymmetry between parties, lack of standardized pricing, difficulty in verifying the credibility of participants, and inefficient communication channels.

EthioGebiya is proposed as an innovative digital marketplace platform that aims to transform the traditional brokerage system by leveraging modern technology. The platform serves as a comprehensive digital ecosystem connecting individual buyers, individual sellers, professional brokers, and registered companies across multiple sectors throughout Ethiopia while preserving and enhancing the valuable role of professional brokers. Rather than eliminating brokers, EthioGebiya empowers them with digital tools that extend their reach, improve efficiency, and build verifiable trust systems.

The platform is accessible across all regions of Ethiopia, from major urban centers like Addis Ababa, Bahir Dar, Dire Dawa, and Hawassa to smaller towns and rural areas where traditional brokerage plays a crucial role in local economies. The system leverages the MERN stack (MongoDB, Express, React, Node.js) for its development, providing a robust, scalable, and modern technology foundation.

1.2 Statement of the Problem

The current brokerage system in Ethiopia suffers from several significant challenges that affect both service providers and end-users across the entire country:

1. Information Asymmetry: Buyers and sellers across Ethiopia often lack access to complete, accurate information about products, prices, and market conditions. Brokers possess information that is not readily available to the public, creating power imbalances in negotiations and potentially leading to unfair transactions. This problem is particularly acute in regions far from major urban centers where market information is even scarcer.

2. Geographical Limitations: Traditional brokers are typically limited to specific localities where they have established networks. A broker in Addis Ababa cannot easily connect with buyers in Bahir Dar, and a seller in Hawassa cannot reach buyers in Dire Dawa without incurring significant costs and inefficiencies. This geographical fragmentation severely limits market opportunities for both buyers and sellers across Ethiopia's diverse regions.

3. Trust and Verification Issues: In the current system, verifying the credibility of brokers, buyers, and sellers is extremely difficult across Ethiopia. There are no standardized mechanisms for background checks, reputation tracking, or dispute resolution, leading to risks of fraud, misrepresentation, and unsatisfactory transactions. This problem is exacerbated by the lack of centralized identification verification systems.

4. Inefficient Communication: Communication between parties relies heavily on phone calls, physical meetings, and word-of-mouth, which is time-consuming, lacks documentation, and often leads to misunderstandings or lost opportunities. In Ethiopia's context, where travel between regions can take days, this inefficiency creates significant barriers to commerce.

5. Lack of Price Transparency: Without centralized information about market prices, buyers may overpay while sellers may underprice their offerings. This inefficiency affects the entire market ecosystem and is particularly problematic in Ethiopia's rapidly changing economic environment where price information is not readily available across regions.

6. Limited Access to Ancillary Services: Users needing additional services such as property inspection, vehicle verification, legal documentation, or financing must independently locate and vet these service providers across Ethiopia's vast geography, adding complexity and risk to transactions.

7. No Transaction Records: The absence of systematic record-keeping makes it difficult to track market trends, analyze pricing patterns, or resolve disputes that may arise after transactions are completed. This lack of documentation also contributes to the informal nature of the economy.

8. Exclusion of Technology-Enabled Participants: Younger, tech-savvy potential users who prefer digital interactions are underserved by the current broker-dependent system, potentially losing economic opportunities. This represents a significant portion of Ethiopia's growing youth population.

9. Limited Reach to Rural Areas: Rural communities across Ethiopia, which constitute a significant portion of the population, have very limited access to organized brokerage services. Farmers, small business owners, and rural residents face particular challenges in connecting with buyers and sellers from other regions.

10. Payment Security Concerns: Cash-based transactions dominate the current system, creating security risks and limiting the ability to conduct large-scale transactions safely. There is no secure mechanism to protect both parties during transactions.

1.3 Literature Review

Existing literature on digital marketplaces and platform economics provides valuable insights for the EthioGebiya project. Studies have examined the transformation of traditional intermediary models through technology in various emerging markets.

Digital Marketplace Evolution: Research indicates that successful digital marketplaces in developing economies share common characteristics: they address clear pain points in existing systems, provide value to all participant groups, build trust through verification mechanisms, and adapt to local contexts rather than simply importing foreign models. Studies from Kenya's M-Pesa and Nigeria's Jumia demonstrate that platforms integrating local payment methods and cultural contexts achieve higher adoption rates.

Broker Role Transformation: Literature on platforms like OLX, Craigslist, and various African e-commerce sites suggests that pure peer-to-peer models that completely eliminate intermediaries often face challenges with trust, quality control, and user adoption. Hybrid models that integrate professional intermediaries with digital tools have shown greater success in markets with established broker traditions. Research on Rwanda's real estate platforms indicates that brokers who adopt digital tools increase their transaction volumes by 40–60%.

Trust Building in Digital Platforms: Studies emphasize that trust is the critical currency of online marketplaces. Platforms that implement robust verification systems, user ratings, transaction escrow, and clear dispute resolution mechanisms achieve higher user adoption and retention rates. Research from South Africa's Property24 and Nigeria's Property Pro shows that platforms with strong verification systems reduce fraud incidents by up to 70%.

Technology Adoption in Ethiopia: Research on technology adoption patterns in Ethiopia indicates rapidly increasing smartphone penetration, growing internet connectivity in urban areas, and rising digital literacy among younger populations. However, successful platforms must accommodate users with varying levels of digital proficiency through multiple access modalities including USSD and SMS for feature phone users who still represent a significant portion of the population.

Marketplace Revenue Models: Analysis of successful marketplace platforms reveals diversified revenue approaches including premium subscriptions and featured listings. Models that align platform success with user success tend to achieve sustainable growth. Research on African marketplace platforms indicates that subscription-based models generate stable and predictable revenue streams.

Mobile Money Integration: Studies on African digital marketplaces highlight the critical importance of mobile money integration. In Ethiopia, the availability of Chapa payment gateway, Telebirr, M-Pesa, and CBE Birr has created new opportunities for digital payments. Platforms that integrate seamlessly with these services achieve significantly higher transaction completion rates.

MERN Stack Applications: Technical literature on the MERN stack (MongoDB, Express, React, Node.js) demonstrates its effectiveness for developing scalable, real-time applications. Case studies show that MERN-based platforms achieve faster development cycles, better performance for real-time features, and easier scalability compared to traditional LAMP stack applications. The stack's JavaScript foundation across frontend and backend reduces development complexity and enables code reuse.

1.4 Objective of the Project

1.4.1 General Objective

The general objective of this project is to develop a comprehensive digital marketplace and brokerage platform called "EthioGebiya" using the MERN stack (MongoDB, Express, React, Node.js) that connects individual buyers, sellers, professional brokers, and registered companies across Ethiopia, enhancing the efficiency, transparency, and accessibility of brokered transactions.

1.4.2 Specific Objectives

To achieve the general objective, the following specific objectives are proposed:

1. To study and analyze the current traditional brokerage system across Ethiopia, including major urban centers and rural areas, to understand its processes, challenges, and opportunities.
2. To develop a secure user registration and authentication system using JWT (JSON Web Tokens) with role-based access control for four distinct user types (individual users, brokers, companies, and administrators), including OTP-based phone verification and national ID / business document upload.
3. To design and implement structured listing templates for 13 product categories — Real Estate, Automotive, Electronics, Home & Furniture, Fashion & Clothing, Sports & Fitness, Books & Media, Pets & Animals, Services, Domestic Jobs, Professional Jobs, Business & Industrial, and Agriculture — with category-specific required fields ensuring comprehensive disclosure.
4. To create an integrated real-time communication system using Socket.io enabling in-platform messaging between users tied to specific listings, with WebSocket-based instant delivery and persistent

message history.

5. To implement a premium subscription system with integration for Chapa payment gateway (supporting Telebirr, M-Pesa, and CBE Birr), featuring tiered pricing based on user role and category, and an admin approval workflow for manual payment verification.
6. To design a responsive user interface using React.js with Bootstrap 5, accessible via web browsers, with USSD integration for users with limited smartphone access across Ethiopia, and full Amharic/English bilingual support via i18next.
7. To support a Company actor type that allows registered businesses to create a company profile, invite team members, and post listings on behalf of the company, with a dedicated verification workflow.
8. To develop administrative dashboards for platform monitoring, user management, listing moderation, company verification, premium subscription management, and dispute resolution through moderation reports.
9. To implement data analytics capabilities using MongoDB aggregation for market insights and platform optimization.
10. To build the system using the MERN stack (MongoDB, Express, React, Node.js) to ensure scalability, performance, and maintainability.

1.5 Scope of the Project

The "EthioGebiya" project encompasses the development of a comprehensive digital marketplace platform with the following scope:

Included Features:

- User registration and profile management for four roles: individual user, broker, company, and administrator
- OTP-based phone verification via AfroMessage SMS gateway
- National ID upload for individual users and brokers (stored on Cloudinary)
- Business registration document upload for company verification
- Structured listing creation for 13 categories with category-specific required fields
- Image upload (up to 10 images, 10 MB each) and video upload (1 video, 50 MB) per listing via Cloudinary
- Premium subscription gate: all non-admin users must hold an active category-level subscription or use their one free trial listing
- Tiered subscription pricing (basic/standard/premium) based on role, category price level (low/medium/high), and duration (monthly/yearly)
- Chapa payment gateway integration (Telebirr, M-Pesa, CBE Birr) for subscription payments
- Advanced search and filtering by keyword (MongoDB full-text), category, price range, region, and geolocation radius
- Saved searches for registered users
- In-platform real-time messaging via Socket.io with file attachments, per-conversation history, and unread tracking

- Notification system (in-app and real-time push via Socket.io) for messages, verification updates, listing approvals, subscription status changes
- Company actor type: company profile, logo/cover photo, team member management (invite/remove, editor/viewer roles)
- Admin panel for user management (activate/suspend), listing moderation (approve/reject), company verification, premium subscription management (activate/reject/deactivate), category management, and activity log review
- Moderation report system (users can report listings or messages; admins can warn, remove, or suspend)
- USSD interface for basic real estate and automotive search on feature phones via Ethio Telecom gateway
- Mobile-responsive web interface (React + Bootstrap 5)
- Bilingual interface (Amharic and English) via i18next
- Redis-backed session management and Socket.io adapter for horizontal scaling
- Automatic listing expiry job (node-cron) with expiry notification

Geographical Scope: Nationwide coverage across all regions of Ethiopia, with initial focus on major urban centers including Addis Ababa, Bahir Dar, Dire Dawa, Hawassa, Mekelle, and Jimma, with plans to expand to all zones and woredas.

Technology Stack

Layer	Technology
Frontend	React.js 19, Bootstrap 5, Vite 8, Redux Toolkit, React Router 7
Backend	Node.js, Express.js 5
Database	MongoDB, Mongoose
Authentication	JWT (access 15 min + refresh 7 days), bcryptjs, Redis token store
Real-time	Socket.io 4, @socket.io/redis-adapter
Payment	Chapa API (Telebirr, M-Pesa, CBE Birr)
Media Storage	Cloudinary
SMS/OTP	AfroMessage (Ethiopian telecom SMS gateway)
USSD	Ethio Telecom USSD gateway
Caching/Sessions	Redis (ioredis)
Internationalisation	i18next, react-i18next
Scheduled Jobs	node-cron (listing expiry)
Security	Helmet, CSRF token, express-mongo-sanitize, express-rate-limit

Out of Scope

- Physical inspection services (third-party referral only)
- Legal documentation preparation (third-party referral only)
- Financing or lending services

- International transactions or cross-border brokerage
- Physical delivery or logistics services
- Full native mobile application development (Phase 2)
- In-app escrow/transaction processing (Phase 2)

1.6 Significance of the Project

The EthioGebiya project holds significant value for multiple stakeholders across Ethiopia:

For Buyers:

- Access to a wider range of products and sellers across all Ethiopian regions
- Transparent pricing information enabling informed decisions
- Verified user and company profiles reducing fraud risks
- Structured, category-specific product information facilitating meaningful comparisons
- Ability to communicate with multiple sellers efficiently via real-time chat
- Secure subscription-based platform incentivizing only serious listers

For Sellers and Brokers:

- Expanded market reach beyond local networks to a national audience
- Digital tools for efficient listing management with image and video uploads
- Reputation building through verified profiles and national ID review
- Reduced time spent on administrative tasks
- Access to market analytics and pricing insights across regions
- Increased transaction volume through broader market access

For Companies:

- Dedicated company profile and branding (logo, cover photo, description)
- Ability to post professional jobs, services, and other categories under the company identity
- Team member management with role-based access (editor/viewer)
- Verified company status badge after admin document review

For the Ethiopian Economy:

- Formalization of informal brokerage activities across the country
- Increased market efficiency and transaction volume
- Job creation in the digital economy, particularly for youth
- Enhanced consumer protection through transparent practices
- Data generation for market analysis and policy development
- Contribution to the Digital Ethiopia 2025 strategy
- Improved economic integration across regions

For Developers:

- Fulfillment of degree requirement
- Practical experience in full-stack MERN development
- Understanding of real-world business requirements
- Contribution to Ethiopian digital transformation

- Hands-on experience with modern technologies including React, Node.js, MongoDB, Redis, Socket.io, and Cloudinary

---

1.7 Tools and Methodology

1.7.1 Data Collection Methodology

To gather requirements and understand the existing brokerage system across Ethiopia, the following data collection methods were employed:

Primary Data Sources:

Interviews: Semi-structured interviews were conducted with:

- 25 traditional brokers in Addis Ababa (real estate and automotive sectors)
- 15 brokers in Bahir Dar
- 12 brokers in Hawassa
- 10 brokers in Dire Dawa
- 30 individual buyers and sellers across various regions
- 10 small business owners who regularly engage brokers
- Representatives from 3 property inspection companies
- 5 representatives from financial institutions (CBE, Dashen, etc.)

Interview questions focused on current processes, challenges, technology usage, willingness to adopt digital tools, expectations from a digital platform, and payment preferences.

Observations: The team observed brokerage activities at:

- Traditional broker offices in Merkato area, Addis Ababa
- Vehicle brokerage centers around Mexico Square and other locations
- Real estate broker operations in Bole, Kazanchis, and other areas
- Broker-client negotiation sessions (with permission)
- Brokerage activities in regional cities to understand regional differences

Observations documented workflow patterns, documentation practices, communication methods, and common pain points.

Secondary Data Sources:

Document Analysis: Review of existing brokerage records, sample contracts, and transaction documents to understand information requirements and documentation standards.

Literature Review: Analysis of academic papers, industry reports, and case studies on digital marketplace implementations in emerging economies, particularly in Africa.

Competitor/Platform Analysis: Study of existing online platforms used in Ethiopia including Facebook Marketplace, Telegram groups, and international classified sites to identify strengths, weaknesses, and gaps.

Government Policy Review: Analysis of Ethiopian government policies regarding digital economy, e-commerce, and financial technology to ensure compliance.

1.7.2 Programming Languages and Technologies

Based on system requirements and team expertise, the following MERN stack technologies were used:

Frontend Development:

- **React.js 19:** For building component-based, responsive user interfaces
- **HTML5/CSS3:** For structuring and styling web pages
- **JavaScript (ES6+):** For client-side interactivity and dynamic content
- **Bootstrap 5:** For responsive design ensuring compatibility across devices
- **Axios:** For HTTP requests to backend API
- **React Router 7:** For client-side routing
- **Redux Toolkit:** For state management
- **i18next / react-i18next:** For Amharic/English bilingual support
- **Socket.io-client 4:** For real-time messaging on the frontend
- **Vite 8:** For fast frontend build tooling

Backend Development:

- **Node.js:** JavaScript runtime for server-side development
- **Express.js 5:** Web framework for building RESTful API
- **Socket.io 4:** For real-time messaging and notifications
- **JWT (JSON Web Tokens):** For secure authentication and authorization (15-minute access tokens, 7-day refresh tokens)
- **Bcryptjs:** For password hashing and encryption (cost factor 12)
- **Multer:** For file uploads (images, documents)
- **express-validator:** For input validation and sanitization
- **express-mongo-sanitize:** For MongoDB injection prevention
- **Helmet:** For HTTP security headers
- **express-rate-limit:** For API and auth rate limiting
- **node-cron:** For scheduled listing expiry jobs

Database:

- **MongoDB:** NoSQL database for flexible, scalable data storage (11 collections)
- **Mongoose 9:** ODM library for MongoDB schema definition and validation
- **MongoDB Atlas:** For cloud database hosting and management

Infrastructure and Integration:

- **Redis (ioredis):** For JWT refresh token storage, session management, and Socket.io adapter
- **Cloudinary:** For image and video storage (listings, profiles, verification docs, company documents)
- **Chapa API:** Ethiopian payment gateway supporting Telebirr, M-Pesa, CBE Birr
- **AfroMessage API:** Ethiopian telecom SMS gateway for OTP delivery
- **Ethio Telecom USSD Gateway:** For basic feature-phone access

1.7.3 System Requirements

Hardware Requirements:

Component	Development Environment	Production Server
Processor	Intel Core i5 or equivalent	Intel Xeon or equivalent
RAM	8 GB minimum	16 GB minimum
Storage	256 GB SSD	500 GB SSD (configurable)
Network	Broadband connection	Dedicated bandwidth

Software Requirements:

Category	Software
Operating System	Windows 10/11, Linux (Ubuntu 20.04+), macOS
Runtime Environment	Node.js 16+
Database	MongoDB 5.0+, MongoDB Atlas
Cache / Session Store	Redis 6+
Development Tools	VS Code, Git, npm, Postman
Testing Tools	Jest, Supertest, fast-check (property-based testing)
Version Control	GitHub/GitLab
Design Tools	Figma, Adobe XD
Diagramming	EdrawMax, Draw.io

1.7.4 System Modeling Tools

For system analysis and design, the following modeling tools and methodologies were used:

UML Diagrams:

- **Use Case Diagrams:** To visualize system functionality from user perspective (created using EdrawMax)
- **Sequence Diagrams:** To illustrate interactions between objects over time
- **Activity Diagrams:** To model workflow processes
- **Class Diagrams:** To represent system structure and relationships
- **Deployment Diagrams:** To show physical architecture

Modeling Software:

- **EdrawMax:** For creating UML diagrams including use case diagrams
- **Draw.io:** For additional diagramming
- **Lucidchart:** For collaborative diagramming
- **MongoDB Compass:** For database modeling and visualization

Development Methodology:

- **Agile/Iterative Approach:** Allowing for continuous improvement based on feedback
- **Sprints:** 2-week development cycles with regular reviews

- **MVP First:** Focus on core functionality before feature expansion

---

1.8 Feasibility Study

1.8.1 Operational Feasibility

The proposed system is operationally feasible for the following reasons:

User Familiarity: Target users (brokers, buyers, sellers, companies) across Ethiopia have increasing exposure to digital platforms through social media and mobile applications, reducing learning curve. The growth of Chapa, Telebirr, and other mobile money services has familiarized many Ethiopians with digital financial transactions.

Process Alignment: The system enhances rather than replaces existing broker roles, reducing resistance to adoption. Brokers will be empowered with digital tools that extend their reach and improve their efficiency.

Multiple Access Modalities: Web and USSD options accommodate varying levels of digital literacy and device access across Ethiopia's diverse population, ensuring inclusivity.

Training Feasibility: Training programs can be developed for professional brokers and companies, with peer-to-peer learning leveraging early adopters. Regional training centers can be established to reach users outside major urban areas.

Support Infrastructure: Customer support systems can be established to assist users during transition, with support available in Amharic and English.

1.8.2 Technical Feasibility

The system is technically feasible because:

Mature Technologies: All proposed technologies (MongoDB, Express, React, Node.js) are well-established with extensive documentation and community support. The MERN stack is widely used for scalable web applications.

Team Competence: Team members have academic training and practical experience with JavaScript, React, Node.js, and MongoDB through coursework and projects.

Scalable Architecture: MongoDB Atlas, Redis, and cloud-based infrastructure allow for scaling as user base grows across Ethiopia. The Socket.io Redis adapter enables horizontal scaling of real-time connections.

Integration Capabilities: APIs exist for Chapa (Telebirr, M-Pesa, CBE Birr), AfroMessage SMS, Cloudinary, and USSD gateway integrations needed for the Ethiopian context.

Security Implementation: Standard security practices (Helmet headers, CSRF tokens, MongoDB injection prevention, JWT authentication, bcrypt password hashing, rate limiting) have been fully implemented.

1.8.3 Economic Feasibility

Cost-Benefit Analysis:

Development Costs (Initial Investment):

Cost Category	Estimated Amount (ETB)
Technology Infrastructure (Cloud, APIs)	400,000
Marketing and User Acquisition	200,000
Operational Expenses	150,000
Contingency Reserve	100,000
Total	**850,000**

Projected Benefits:

Year	Projected Revenue (ETB)
Year 1	3,000,000
Year 2	15,000,000
Year 3	40,000,000

Tangible Benefits:

- Revenue from tiered premium subscriptions (basic/standard/premium across 13 categories with monthly/yearly billing)
- Cost savings compared to traditional brokerage marketing and operations
- Scalable revenue model with minimal marginal cost per user

Intangible Benefits:

- Market efficiency improvement across Ethiopian regions
- Enhanced transparency and trust in commercial transactions
- Formalization of informal sector
- Job creation in digital economy
- Contribution to Ethiopian digital transformation

Break-Even Analysis: The project is projected to achieve operational break-even within 18 months of launch, with cumulative profitability by the end of Year 2.

CHAPTER TWO: SYSTEM ANALYSIS

2.1 Overview of the Existing System

The current brokerage system in Ethiopia operates primarily through informal, manual processes across the country. Based on our research conducted in multiple regions including Addis Ababa, Bahir Dar, Hawassa, and Dire Dawa, the existing system functions as follows:

1. Broker Operations:

- Brokers build personal networks through years of relationship building within specific communities
- Information about available properties or vehicles is maintained in notebooks, files, or mentally
- Client inquiries are handled through phone calls and face-to-face meetings

- Commissions are negotiated on a case-by-case basis, typically 2–5% of transaction value
- No formal training or certification requirements exist
- Reputation is built through word-of-mouth within limited geographic areas
- Many brokers operate without any formal business registration
- Information sharing between brokers in different regions is virtually non-existent

2. Buyer Experience:

- Buyers contact multiple brokers individually to express requirements
- Information received is often inconsistent and incomplete
- Property or vehicle visits require coordination with brokers, often involving significant travel
- Price negotiations occur without market benchmarks
- Payment typically involves cash or bank transfer without any protection mechanism
- Post-purchase support is generally unavailable
- Buyers from other regions face particular challenges in verifying listings

3. Seller Experience:

- Sellers rely on broker networks to find buyers
- Listing a property or vehicle means informing known brokers
- No control over how information is presented to potential buyers
- Difficulty tracking broker performance and buyer interest
- Payment receipt depends on broker follow-through
- Sellers in rural areas have extremely limited access to urban buyers

4. Information Management:

- No centralized database of available properties or vehicles
- Duplicate listings across multiple brokers
- Pricing information is fragmented and inconsistent
- Historical transaction data is rarely maintained
- Market trend analysis is impossible without organized data
- Regional price differences are not documented or understood

5. Communication Flow:

- Primarily phone-based with multiple calls required
- No documentation of conversations or agreements
- Misunderstandings common due to lack of written records
- Time zone and availability constraints limit efficiency
- Communication between regions is expensive and inefficient

6. Trust and Verification:

- No mechanism to verify broker credentials or track record
- Buyers rely on referrals and personal impressions
- Fraud incidents are difficult to prevent or pursue
- Dispute resolution depends on informal community mechanisms
- National ID verification is not available to verify user identities

7. Payment Systems:

- Primarily cash-based with associated security risks
- Bank transfers are possible but require trust between parties
- No protection mechanism available
- Mobile money services like Telebirr are increasingly used but not integrated into the brokerage workflow
- Large transactions require physical presence for cash handling

---

2.2 System Requirement Specification

The EthioGebiya system requirements are categorized into functional and non-functional requirements as follows:

2.3 Functional Requirements

User Management

ID	Requirement Description
FR-01	The system shall allow users to register as User (individual buyer/seller), Broker, or Company
FR-02	The system shall collect and validate user information including full name, phone number, password, and optionally date of birth
FR-03	The system shall implement phone verification via OTP for all new registrations using the AfroMessage SMS gateway; where no SMS gateway is configured, phones shall be auto-verified
FR-04	The system shall allow individual users and brokers to upload a national ID document for identity verification
FR-05	The system shall support broker registration with business license number and license document upload
FR-06	The system shall support company registration with company name, industry, contact details, business registration number, and registration document upload
FR-07	The system shall allow users to update their profile information including display name, location, bio, preferred language, and profile photo
FR-08	The system shall provide password change functionality (current password required) and password reset via OTP
FR-09	The system shall maintain user activity logs for audit purposes (auto-expire after 90 days)
FR-10	The system shall enforce a minimum age of 18 years at registration
FR-11	The system shall enforce a unique phone number per account across the entire platform

Listing Management

ID	Requirement Description
FR-12	The system shall allow users, brokers, companies, and admins to create new listings
FR-13	The system shall provide 13 category-specific listing templates: Real Estate, Automotive, Electronics, Home & Furniture, Fashion & Clothing, Sports & Fitness, Books & Media, Pets & Animals, Services, Domestic Jobs, Professional Jobs, Business & Industrial, and Agriculture
FR-14	The system shall require category-specific mandatory fields for each listing type (e.g., propertyType, sizeSqm for real estate; make, model, year for automotive)
FR-15	The system shall require at least one image or video per listing
FR-16	The system shall allow up to 10 images (max 10 MB each) and 1 video (max 50 MB) per listing, stored on Cloudinary
FR-17	The system shall restrict certain categories to brokers/companies only (domestic-jobs, services) and others to companies only (professional-jobs)
FR-18	The system shall allow listing editing and updates by the owner
FR-19	The system shall support listing status management: activate, deactivate, and renew (expired listings only)
FR-20	The system shall prevent duplicate listings by the same user (same title and price on an active listing)
FR-21	The system shall auto-expire listings 30 days after creation and send expiry notifications
FR-22	The system shall allow owners to toggle featured status on their listings
FR-23	The system shall require a valid active premium subscription (or one-time free trial) before a non-admin user can post a listing in a given category

Search and Discovery

ID	Requirement Description
FR-24	The system shall provide full-text search functionality by keywords using MongoDB text indexes
FR-25	The system shall support filtering by category, price range (min/max), region, city, and sub-category
FR-26	The system shall support geolocation-based search with a configurable radius using MongoDB 2dsphere indexes

ID	Requirement Description
FR-27	The system shall display only approved, active listings in search results
FR-28	The system shall support sorting by relevance, date, and price
FR-29	The system shall allow registered users to save search parameters (saved searches)
FR-30	The system shall show related listings from the same seller on listing detail pages
FR-31	The system shall track and increment view count on each listing fetch

Communication

ID	Requirement Description
FR-32	The system shall provide in-platform real-time messaging between users via Socket.io WebSockets
FR-33	The system shall organize messages into conversations linked to specific listings
FR-34	The system shall support message attachments (images/files) uploaded to Cloudinary
FR-35	The system shall track read/unread status per message
FR-36	The system shall deliver in-app and real-time push notifications for new messages
FR-37	The system shall allow users to block/unblock other users; blocked users cannot send messages

Premium Subscription and Payment

ID	Requirement Description
FR-38	The system shall offer tiered premium subscriptions: basic (individual users), standard (brokers), premium (companies)
FR-39	The system shall calculate subscription fees based on role tier, category price level (low/medium/high), and duration (monthly/yearly)
FR-40	The system shall integrate with Chapa payment gateway for online payments (Telebirr, M-Pesa, CBE Birr)

ID	Requirement Description
FR-41	The system shall support manual payment submission (receipt upload) with admin approval workflow
FR-42	The system shall notify users of subscription activation, rejection, expiry (3-day reminder and on-expiry)
FR-43	The system shall allow each user one free trial listing before requiring a subscription
FR-44	The system shall allow admins to activate, deactivate, and reject premium subscriptions

Company Features

ID	Requirement Description
FR-45	The system shall allow company registration with a dedicated company profile (name, industry, logo, cover photo, description, social links)
FR-46	The system shall support company verification by admin via business registration document review
FR-47	The system shall allow company admins to invite team members with editor or viewer roles
FR-48	The system shall allow team members to accept, decline, or be removed from a company
FR-49	The system shall display a public company profile page showing company details and its active listings

Administration

ID	Requirement Description
FR-50	The system shall provide an admin dashboard for platform monitoring
FR-51	The system shall allow admin to manage user accounts (activate, suspend) and revoke sessions
FR-52	The system shall enable admin to review and approve/reject listings with optional rejection reason
FR-53	The system shall enable admin to verify/reject individual users (national ID review) and brokers (license review)
FR-54	The system shall enable admin to verify/reject company registrations

ID	Requirement Description
FR-55	The system shall support moderation report management (listings and messages): admin can warn owner, remove listing, suspend owner, or dismiss
FR-56	The system shall generate platform usage statistics (user counts by role, listing counts by status/category, subscription revenue) using MongoDB aggregation
FR-57	The system shall allow admin to manage categories (add, edit, activate/deactivate, configure required fields)
FR-58	The system shall auto-seed a default admin account on server startup if none exists
FR-59	The system shall auto-seed 13 default categories on server startup if they do not exist

USSD

ID	Requirement Description
FR-60	The system shall provide a USSD interface for Ethio Telecom feature phones allowing users to search real estate and automotive listings by region

2.4 Non-Functional Requirements

Usability

ID	Requirement Description
NFR-01	The system shall have an intuitive, user-friendly interface
NFR-02	The system shall be accessible in Amharic and English languages via i18next internationalization
NFR-03	The system shall provide clear error messages and guidance
NFR-04	The system shall load within 3 seconds on standard connections
NFR-05	The system shall be responsive across desktop, tablet, and mobile devices using Bootstrap 5
NFR-06	The system shall provide a USSD interface for basic functionality on feature phones

Security

ID	Requirement Description
NFR-07	All user passwords shall be encrypted using bcryptjs with cost factor 12
NFR-08	All sensitive data transmission shall use SSL/TLS encryption (HTTPS)

ID	Requirement Description
NFR-09	The system shall implement role-based access control using JWT (roles: user, broker, company, admin)
NFR-10	Access tokens shall expire after 15 minutes; refresh tokens after 7 days, stored in Redis
NFR-11	The system shall protect against MongoDB injection (express-mongo-sanitize), XSS, and CSRF attacks (Helmet, CSRF token middleware)
NFR-12	The system shall apply rate limiting on authentication endpoints and all API routes
NFR-13	User data shall be stored with appropriate privacy protections
NFR-14	Verification documents (national ID, license, registration docs) shall be stored securely on Cloudinary

Reliability

ID	Requirement Description
NFR-15	The system shall have 99.5% uptime during business hours
NFR-16	The system shall perform automated daily backups via MongoDB Atlas
NFR-17	The system shall have failover mechanisms for critical components
NFR-18	Transaction and subscription data shall never be lost once committed
NFR-19	The system shall recover from failures within 2 hours

Performance

ID	Requirement Description
NFR-20	The system shall support up to 10,000 concurrent users
NFR-21	Search results shall return within 2 seconds
NFR-22	Page loads shall complete within 3 seconds
NFR-23	The system shall handle peak loads during business hours
NFR-24	Database queries shall be optimized using MongoDB indexes (2dsphere, text, compound, TTL)

Scalability

ID	Requirement Description
NFR-25	The system architecture shall support horizontal scaling (Redis adapter for Socket.io, MongoDB Atlas)

ID	Requirement Description
NFR-26	MongoDB database design shall accommodate growth to 1 million users
NFR-27	The system shall support addition of new categories and verticals via the admin category management interface
NFR-28	API design shall support future mobile application development

Maintainability

ID	Requirement Description
NFR-29	Code shall follow consistent naming conventions and documentation standards
NFR-30	The system shall include comprehensive error logging
NFR-31	System components shall be modular for easy updates (MVC pattern: models, controllers, routes, services)

2.5 Overview of the Proposed System

The proposed EthioGebiya system is a comprehensive digital marketplace platform built on the MERN stack (MongoDB, Express, React, Node.js) designed to transform traditional brokerage across Ethiopia. The system architecture incorporates multiple layers serving different user needs with a focus on scalability, security, and accessibility.

System Architecture Overview:

The system follows a modern three-tier architecture:

- **Presentation Tier:** React.js 19 frontend providing responsive web interfaces, built with Vite and served as a SPA
- **Application Tier:** Node.js/Express.js 5 backend handling business logic and REST API services, with Socket.io for real-time features
- **Data Tier:** MongoDB for flexible, scalable data storage; Redis for caching and session management

User-Facing Features:

For All Users:

- Bilingual interface (Amharic/English) via i18next
- Secure registration and login with JWT authentication (access + refresh token pair)
- Profile management with national ID verification workflow
- Search and discovery tools with category, price, region, and geolocation filtering
- Real-time messaging system using Socket.io
- In-app notification system with real-time delivery

- Mobile-responsive design for all devices

For Individual Users (Buyers/Sellers):

- Browse and search listings across all 13 categories
- Post listings in general consumer categories (real estate, automotive, electronics, etc.) with one free trial then premium subscription required
- Contact sellers via real-time in-platform chat
- Save search criteria for reuse
- Report suspicious listings or messages

For Brokers:

- All user features plus broker-exclusive categories (domestic-jobs)
- National ID + business license document upload for verification
- Standard-tier subscription pricing
- Ability to manage multiple listings across categories
- Verified broker badge after admin approval

For Companies:

- Dedicated company profile with branding (logo, cover photo, social links, description)
- Company-only categories (professional-jobs, services)
- Premium-tier subscription pricing
- Team member management: invite users as editors or viewers
- Public company profile page listing all active company listings
- Verified company badge after admin document review

For Administrators:

- User management dashboard: search, suspend, reinstate, verify/reject users and brokers
- Listing moderation: approve/reject new listings with rejection reasons
- Company verification: review registration documents, approve/reject companies
- Premium subscription management: activate/reject/deactivate subscriptions
- Moderation report queue: review user reports, take actions (warn, remove, suspend, dismiss)
- Category management: create, edit, activate/deactivate categories and their required fields
- Platform analytics: user counts by role, listing counts by status and category, subscription revenue

Core System Components:

1. User Management Module (Node.js/Express)

- Handles registration, OTP verification, authentication, profile management, and password reset
- JWT-based authentication with refresh tokens stored in Redis
- Role-based access control (RBAC) for four user types: user, broker, company, admin
- Multi-level verification: phone OTP, national ID upload, business license upload
- Secure password storage using bcryptjs (cost factor 12)

2. Listing Management Module (React/Node.js)

- Manages creation, editing, categorization, display, and deletion of listings

- 13 category-specific templates with validated required fields
- Image and video upload to Cloudinary (max 10 images, 1 video per listing)
- Listing status management: active, inactive, expired, removed
- Automatic 30-day expiry with cron job and expiry notification
- Premium subscription gate: subscription check before listing creation

3. Search Engine Module (MongoDB/Node.js)

- Full-text search using MongoDB text indexes on title and description
- Advanced filtering by category, price range, region, city, sub-category
- Geolocation-based search using MongoDB 2dsphere indexes
- Sorting by relevance (text score), date (createdAt), price
- Saved search functionality for registered users

4. Communication Module (Socket.io/Node.js)

- Real-time in-platform messaging via WebSocket connections
- Conversations linked to specific listings
- Message history stored in MongoDB
- File attachment support (uploaded to Cloudinary)
- Read/unread tracking per message
- User blocking to prevent unwanted messages

5. Premium Subscription Module (Node.js)

- Tiered pricing: basic (user), standard (broker), premium (company)
- Category price levels: low (electronics, fashion, etc.), medium (services, jobs), high (real estate, automotive, business)
- Duration options: monthly and yearly (10-month discount on yearly)
- Chapa payment gateway integration for online payment (Telebirr, M-Pesa, CBE Birr)
- Manual payment flow: receipt image upload + admin approval
- Subscription lifecycle: pending → active → expired
- Expiry job: daily check, 3-day reminder and expiry notification

6. Company Module (React/Node.js)

- Company profile creation with branding assets (Cloudinary-hosted logo and cover photo)
- Business registration document upload and admin verification workflow
- Team member management with invite/accept/decline lifecycle (editor/viewer roles)
- Public company profile page with active listings

7. Administration Module (React/Node.js)

- Full user management with search, pagination, suspend/reinstate
- Listing moderation queue with approve/reject actions
- Company verification queue
- Premium subscription queue (pending → active/rejected)
- Moderation report resolution workflow
- Category management with dynamic required fields configuration
- Platform analytics dashboard

Technology Integration Points:

- **MongoDB Atlas:** Cloud database with automated backups and scaling
- **Redis:** For JWT refresh token storage, session management, and Socket.io horizontal scaling adapter
- **Cloudinary:** For all media storage — listing images/videos, profile photos, verification documents, company assets
- **AfroMessage API:** Integration with Ethiopian telecom for OTP SMS delivery
- **Chapa API:** Ethiopian payment gateway supporting Telebirr, M-Pesa, CBE Birr
- **Ethio Telecom USSD Gateway:** For basic feature-phone access to real estate and automotive searches

---

2.6 Business Rule Identification

Based on analysis of the brokerage domain and system requirements, the following business rules govern EthioGebiya operations:

ID	Business Rule	Rationale
BR-01	Each user must have a unique registered phone number	Prevents duplicate accounts and enables verification
BR-02	Users must verify phone number via OTP within 24 hours of registration (where SMS gateway is available)	Ensures genuine user contact information
BR-03	Individual users must submit a valid national ID image for verification	Establishes identity and builds trust
BR-04	Brokers must submit a valid business license and national ID for premium verification	Establishes professional credibility
BR-05	Companies must submit a valid business registration document for verification	Establishes corporate credibility
BR-06	Listings must include all mandatory category-specific fields	Ensures complete information for buyers
BR-07	At least one image or video is required per listing	Ensures visual proof of product/service
BR-08	Users must be at least 18 years old to register	Legal capacity for transactions
BR-09	Non-admin users must hold an active premium subscription for the relevant category, or use their one-time free trial, before creating a listing	Platform revenue model and quality control
BR-10	Certain categories (domestic-jobs) are restricted to brokers and companies only	Category appropriateness by actor type
BR-11	Certain categories (professional-jobs, services) are restricted to companies only	Category appropriateness by actor type

ID	Business Rule	Rationale
BR-12	Listings are auto-approved for verified brokers and companies; admin review may still occur via moderation reports	Streamlines posting for trusted actors
BR-13	Duplicate listings by same user (same title and price, active status) are rejected with a warning	Prevents spam and misuse
BR-14	Listings automatically expire 30 days after creation	Ensures listing freshness and relevance
BR-15	Subscription fees are calculated from: role tier × category price level × duration	Transparent, predictable pricing
BR-16	Yearly subscriptions are priced as 10 months (2-month discount vs monthly)	Incentivizes long-term commitment
BR-17	Chapa payments are verified via the Chapa verification API before activation	Ensures payment security
BR-18	Admin accounts cannot be suspended by other admins	Protects platform integrity
BR-19	Users cannot suspend their own account	Prevents accidental self-suspension
BR-20	Blocked users cannot send messages to the user who blocked them	Privacy and harassment protection

2.7 System Requirement Analysis

System requirement analysis involved thorough examination of the gathered requirements to ensure completeness, consistency, and feasibility:

Requirement Validation:

- Each functional requirement was reviewed against user needs identified during interviews across Ethiopia
- Requirements were prioritized using MoSCoW method (Must have, Should have, Could have, Won't have)
- Technical feasibility was assessed for each requirement using MERN stack capabilities
- Dependencies between requirements were identified
- Conflicts were resolved through stakeholder consultation

Requirement Categorization:

Priority	Count	Description
Must Have	35	Core functionality essential for MVP launch
Should Have	15	Important features for Phase 2
Could Have	8	Desirable features for future releases

Priority	Count	Description
Won't Have	5	Features beyond current scope

Key Findings:

- User and company verification with document upload emerged as the highest priority concern across all stakeholder groups
- Mobile accessibility is critical for the Ethiopian context, especially outside major urban areas
- A tiered premium subscription model (role × category × duration) better reflects the diversity of use cases than a single flat subscription
- A Company actor type was essential to support businesses that need to post professional jobs and services under their brand identity
- USSD access is needed for users without smartphones
- The one-free-trial mechanism lowers the barrier to entry while sustaining the revenue model

---

2.8 Actors and Use Case Identification

Actors

Based on system analysis, the following actors have been identified:

Table 1:

Actor	Description	Responsibilities
Guest	Unregistered visitor	View public listings, search products, view company profiles
User	Registered individual buyer/seller	Search products, contact sellers, create listings (with subscription), manage profile
Broker	Registered professional intermediary	All user functions plus broker-exclusive categories, business license submission
Company	Registered business entity	Company profile management, team member management, company-category listings
Administrator	System admin	Manage users, moderate content, verify companies and brokers, manage subscriptions, resolve disputes, view reports
Payment Gateway	External system (Chapa / Telebirr / M-Pesa / CBE Birr)	Process payments, provide transaction verification
SMS Gateway	External system (AfroMessage)	Send OTP verification codes and notifications
USSD Gateway	External system (Ethio Telecom)	Relay USSD sessions for feature phone users

Use Case Identification

ID	Use Case Name	Primary Actor	Description
UC-01	Login	All Users	Authenticate and access system
UC-02	Register (User)	Guest	Create new individual user account
UC-03	Register (Broker)	Guest	Create broker account with license upload
UC-04	Register (Company)	Guest	Create company account with registration doc upload
UC-05	Verify OTP	All Users	Complete phone verification
UC-06	Manage Profile	All Users	Update personal information, photo, language
UC-07	Change Password	All Users	Update account password
UC-08	Reset Password	All Users	Reset password via OTP
UC-09	Upload ID / License	User, Broker	Submit verification documents
UC-10	Create Listing	User, Broker, Company, Admin	Post new product/service listing
UC-11	Edit Listing	User, Broker, Company, Admin	Update existing listing
UC-12	Delete Listing	User, Broker, Company, Admin	Remove listing from platform
UC-13	Update Listing Status	User, Broker, Company	Activate, deactivate, or renew listing
UC-14	Toggle Featured Listing	User, Broker, Company	Mark/unmark listing as featured
UC-15	Search Products	Guest, All Users	Find products by criteria and filters
UC-16	View Listing Details	Guest, All Users	See complete product information
UC-17	Save Search	User, Broker, Company	Save search parameters for reuse

ID	Use Case Name	Primary Actor	Description
UC-18	Send Message	User, Broker, Company	Exchange messages in a conversation
UC-19	View Conversations	All Users	Read conversation history
UC-20	Block/Unblock User	All Users	Manage blocked contacts
UC-21	Subscribe Premium	User, Broker, Company	Purchase category-level subscription
UC-22	Verify Chapa Payment	User, Broker, Company	Confirm online payment after redirect
UC-23	Upload Manual Payment	User, Broker, Company	Submit receipt for admin approval
UC-24	Manage Company	Company	Update company profile and branding
UC-25	Manage Team Members	Company	Invite, view, remove team members
UC-26	View Company Profile	Guest, All Users	See public company page with listings
UC-27	Report Content	All Users	Report listing or message for moderation
UC-28	Manage Users	Administrator	Search, suspend, reinstate user accounts
UC-29	Verify Users/Brokers	Administrator	Approve or reject identity verification
UC-30	Moderate Listings	Administrator	Approve or reject submitted listings
UC-31	Verify Companies	Administrator	Approve or reject company registrations
UC-32	Manage Subscriptions	Administrator	Activate, deactivate, reject subscriptions
UC-33	Resolve Reports	Administrator	Handle moderation reports
UC-34	Manage Categories	Administrator	Add, edit, configure category required fields
UC-35	View Reports	Administrator	Access platform analytics and statistics

ID	Use Case Name	Primary Actor	Description
UC-36	Access via USSD	Feature Phone Users	Use basic search on feature phones

2.9 Use Case Diagram

Based on the EdrawMax diagram provided, the use case diagram for EthioGebiya is as follows:

Figure 1: Use Case Diagram

2.10 Use Case Description

Table 2: Login Use Case Description

Element	Description
Use Case ID	UC-01
Use Case Name	Login
Actor	All Users (User, Broker, Company, Administrator)
Description	User authenticates to access personalized system features
Pre-condition	User must have a registered account with phone verified
Post-condition	User successfully logged in, JWT access and refresh tokens issued, redirected to appropriate page

Basic Flow:

User Action	System Response
1. User navigates to login page	2. System displays login form
3. User enters phone number and password	
4. User clicks Login button	5. System validates credentials
	6. System checks account active status
	7. System checks phone verification status
	8. System generates JWT access token (15 min) and refresh token (7 days), stores refresh token in Redis
	9. System returns tokens and user profile (id, phone, fullName, role, verificationStatus)
	10. Use case ends

Alternative Flow:

Condition	System Response
Invalid phone/password	System displays generic error: "Invalid phone number or password."
Account suspended	System displays generic error message (no enumeration)
Phone not verified	System returns 403 with requiresVerification flag and redirects to OTP verification

Table 3: User Registration Use Case Description

Element	Description
Use Case ID	UC-02
Use Case Name	Register (User)
Actor	Guest
Description	New individual user creates an account on the platform
Pre-condition	User has a valid phone number, is 18+ years old, not previously registered
Post-condition	User account created; phone verified (auto if no SMS gateway, or via OTP)

Basic Flow:

User Action	System Response
1. User selects Register option	2. System displays registration form
3. User enters full name, phone, password, date of birth	
4. User optionally uploads national ID image	
5. User selects role: User	
6. User submits form	7. System validates input data and age (18+)
	8. System checks for duplicate phone number
	9. System hashes password using bcryptjs (cost 12)
	10. System creates account
	11. If SMS gateway configured: System sends OTP via AfroMessage and displays OTP screen
	12. If no SMS gateway: System auto-verifies phone and notifies admins

User Action	System Response
	13. Use case continues with Verify OTP if applicable

Alternative Flow:

Condition	System Response
Missing required fields	System highlights missing fields
Duplicate phone	System returns 409: "An account with this phone number already exists."
Password less than 8 characters	System displays password requirements
Age under 18	System returns 422: "You must be at least 18 years old to register."
SMS send failure	System rolls back user creation and returns 503

Table 4: Verify Account Use Case Description

Element	Description
Use Case ID	UC-05
Use Case Name	Verify OTP
Actor	All Users
Description	User completes phone verification via OTP sent by AfroMessage
Pre-condition	User registered, OTP sent to phone, verification pending
Post-condition	Phone verified (isPhoneVerified = true), user can log in

Basic Flow:

User Action	System Response
1. User receives OTP via SMS	
2. User enters OTP in verification screen	3. System validates OTP against Redis-stored value
	4. System updates isPhoneVerified to true
	5. System invalidates OTP
	6. System notifies admins of new registration
	7. System displays success message
	8. Use case ends

Alternative Flow:

Condition	System Response
Incorrect OTP	System returns 400: "Invalid or expired OTP."
OTP expired	System offers resend OTP option
National ID pending	Admin receives verification_pending notification for manual review

Table 5: Search Products Use Case Description

Element	Description
Use Case ID	UC-15
Use Case Name	Search Products
Actor	Guest, All Users
Description	User searches for listings using various criteria
Pre-condition	None
Post-condition	Search results displayed (approved, active listings only)

Basic Flow:

User Action	System Response
1. User enters search terms in search box	
2. User selects category/region filter	
3. User sets price range	
4. User clicks Search button	5. System processes search query against MongoDB text index
	6. System retrieves matching approved, active listings
	7. System displays results with title, price, location, primary image
	8. System offers filtering and sorting options
9. User applies additional filters (city, minPrice, maxPrice, geolocation radius)	10. System updates results
11. User sorts results	12. System reorders display
	13. Use case continues until user exits

Alternative Flow:

Condition	System Response
No results found	System displays "No results" message
Geolocation filter	System applies 2dsphere geospatial filtering

Table 6: Send Message Use Case Description

Element	Description
Use Case ID	UC-18
Use Case Name	Send Message
Actor	User, Broker, Company
Description	Users communicate via real-time in-platform messaging linked to a listing
Pre-condition	Both users registered and logged in; sender has not been blocked by recipient
Post-condition	Message stored in MongoDB, delivered in real-time via Socket.io

Basic Flow:

User Action	System Response
1. User clicks "Contact" on listing	2. System opens or retrieves existing conversation for that listing
3. System establishes Socket.io WebSocket connection	
4. User types message	
5. User optionally attaches file	6. System validates attachment and uploads to Cloudinary
7. User clicks Send	8. System stores message in MongoDB
	9. System emits real-time event to recipient via Socket.io
	10. System creates in-app notification for recipient
	11. System updates conversation's lastMessage reference
	12. Use case ends

Alternative Flow:

Condition	System Response
Recipient offline	Message stored in MongoDB, delivered when recipient reconnects
Attachment upload fails	System returns error, message not sent

Condition	System Response
Sender is blocked by recipient	System prevents message delivery

Table 7: Manage Users Use Case Description

Element	Description
Use Case ID	UC-28
Use Case Name	Manage Users
Actor	Administrator
Description	Admin oversees and manages user accounts
Pre-condition	Admin logged in with appropriate privileges
Post-condition	User accounts managed; session revoked on suspension

Basic Flow:

User Action	System Response
1. Admin navigates to User Management	2. System displays paginated user list with search
3. Admin searches/filters for specific user (phone, name, or ID)	4. System updates display
5. Admin selects user	6. System displays user details
7. Admin selects action (Suspend/Reinstate)	8. System requests confirmation
9. Admin confirms	10. System updates user isActive status in MongoDB
	11. If suspended: System deletes Redis refresh token, revokes all sessions
	12. System sends suspension/reinstatement notification to user
	13. Use case ends

Alternative Flow:

Condition	System Response
Attempt to suspend admin account	System returns 403: "Admin accounts cannot be suspended."
Attempt to suspend own account	System returns 403: "You cannot suspend your own account."

Table 8: Process Payment Use Case Description

Element	Description
Use Case ID	UC-21/22
Use Case Name	Subscribe Premium (Chapa Payment)
Actor	User, Broker, Company, Payment Gateway (Chapa)
Description	User purchases a category-level premium subscription via Chapa payment gateway
Pre-condition	User logged in; subscription record created with status "pending"
Post-condition	Payment verified, subscription activated; listing creation unlocked for that category

Basic Flow:

User Action	System Response
1. User selects category and duration (monthly/yearly) on subscription page	2. System calculates fee from PRICE_TABLE (role tier × category level × duration)
3. User selects payment method (Telebirr/M-Pesa/CBE Birr via Chapa)	4. System creates pending subscription record in MongoDB
	5. System initializes Chapa transaction and returns checkout URL
6. User completes payment on Chapa-hosted page	7. Chapa processes payment and redirects user to return URL
8. User lands on verification page with txRef	9. System calls Chapa verify API with txRef
	10. System checks transaction status
	11. On success: System activates subscription, sets startDate and expiryDate
	12. System notifies user of activation
	13. Use case ends

Alternative Flow:

Condition	System Response
Payment declined/failed	System marks subscription as inactive, notifies user
Manual payment (receipt upload)	User uploads receipt image; admin reviews and manually activates or rejects
Subscription already active	System returns error: subscription already exists for that category

Table 9: View Reports Use Case Description

Element	Description
Use Case ID	UC-35
Use Case Name	View Reports
Actor	Administrator
Description	Admin accesses platform analytics and statistics
Pre-condition	Admin logged in with appropriate permissions
Post-condition	Reports displayed

Basic Flow:

User Action	System Response
1. Admin navigates to Admin Panel	2. System displays analytics dashboard
3. Admin selects report type (users by role, listings by status/category, subscriptions)	
	4. System generates aggregated statistics using MongoDB aggregation pipelines
	5. System displays data in tables and charts
6. Use case ends	

Alternative Flow:

Condition	System Response
No data for period	System displays empty state with message

2.11 Sequence Diagram

Figure 2: Sequence Diagram for User Registration

Figure 3: Sequence Diagram for Post Listing

Figure 4: Sequence Diagram for Search Products

Figure 5: Sequence Diagram for Message Exchange

Figure 6: Sequence Diagram for Process Payment (Premium Subscription — Chapa)

2.12 Activity Diagram

Figure 8: Activity Diagram for Login

Figure 9: Activity Diagram for User Registration

Figure 10: Activity Diagram for Post Listing

Figure 11: Activity Diagram for Search Products

Figure 12: Activity Diagram for Premium Subscription

Figure 13: Activity Diagram for Message Exchange

2.13 Analysis Class Diagram

Figure 14: Analysis Class Diagram

CHAPTER THREE: SYSTEM DESIGN

3.1 Design Class Diagram

Figure 15: Design Class Diagram

3.2 Physical Data Model

Figure 16: Physical Data Model (MongoDB Collections)

The EthioGebiya database uses MongoDB with 11 collections. Below is the complete schema for each collection.

Table 10: Users Collection Structure

Field	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY, auto-generated	Unique identifier
phone	String	REQUIRED, UNIQUE, INDEX	User's phone number (primary identifier)
fullName	String	REQUIRED	User's full name
passwordHash	String	REQUIRED	Bcryptjs encrypted password (cost 12)
role	String (enum)	REQUIRED: user, broker, company, admin	Account role
isPhoneVerified	Boolean	DEFAULT false	Phone OTP verification status
verificationStatus	String (enum)	DEFAULT 'none': none, pending, verified, rejected	KYC document verification status
verificationDoc	String		Cloudinary URL to national ID image
businessLicense	String		Broker business license number

Field	Data Type	Constraints	Description
businessLicenseDoc	String		Cloudinary URL to broker license document
licenseType	String		Type of broker license
specializations	[String]		Broker specialization categories
email	String		Optional email address
website	String		Optional website URL
dateOfBirth	Date		Date of birth (age 18+ validation)
profilePhoto	String		Cloudinary URL to profile photo
location	String		Free-text location
city	String		City
region	String		Region
bio	String		Profile bio
preferredLanguage	String (enum)	DEFAULT 'en': en, am	Preferred UI language
isActive	Boolean	DEFAULT true	Account active status
blockedUsers	[ObjectId]	ref: User	List of blocked user IDs
premiumSubscription	Object	embedded	Embedded subscription summary (legacy)
hasUsedFreeTrial	Boolean	DEFAULT false	Whether the one free trial listing has been used
loginAttempts	Number	DEFAULT 0	Failed login attempt counter
lockUntil	Date		Account lock expiry timestamp
settings	Object	embedded	Notification, privacy, and preference settings
createdAt	Date	auto	Account creation timestamp
updatedAt	Date	auto	Last update timestamp

Table 11: Listings Collection Structure

Field	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY, auto-generated	Unique listing identifier

Field	Data Type	Constraints	Description
owner	ObjectId	REQUIRED, INDEX, ref: User	Reference to listing owner
title	String	REQUIRED	Listing title
description	String		Detailed description
category	ObjectId	REQUIRED, ref: Category	Reference to category document
categorySlug	String (enum)	13 valid slugs	Category slug for fast filtering
price	Number	REQUIRED	Price in ETB
currency	String	DEFAULT 'ETB'	Currency code
location.region	String	REQUIRED	Ethiopian region
location.city	String		City
location.address	String		Street address
location.coordinates	GeoJSON Point	2dsphere INDEX	GPS coordinates for geosearch
images	[String]		Array of Cloudinary image URLs
video	String		Cloudinary video URL
status	String (enum)	DEFAULT 'active': active, inactive, expired, removed	Listing operational status
approvalStatus	String (enum)	DEFAULT 'pending': pending, approved, rejected	Admin moderation status
rejectionReason	String		Rejection reason from admin
isFeatured	Boolean	DEFAULT false	Featured listing flag
viewCount	Number	DEFAULT 0	Total view count
expiresAt	Date		Auto-expiry date (30 days from creation)
realEstate	Object		Real estate specific fields (propertyType, sizeSqm, bedrooms, etc.)
automotive	Object		Automotive specific fields (make, model, year, mileage, etc.)
electronics	Object		Electronics specific fields
homeFurniture	Object		Home & Furniture specific fields
fashionClothing	Object		Fashion & Clothing specific fields

Field	Data Type	Constraints	Description
sportsFitness	Object		Sports & Fitness specific fields
booksMedia	Object		Books & Media specific fields
petsAnimals	Object		Pets & Animals specific fields
services	Object		Services specific fields
domesticJobs	Object		Domestic jobs specific fields (jobType, salary, etc.)
professionalJobs	Object		Professional jobs specific fields (jobTitle, industry, etc.)
businessIndustrial	Object		Business & Industrial specific fields
agriculture	Object		Agriculture specific fields
ownerType	String (enum)	DEFAULT 'individual': individual, company	Ownership type
companyOwner	ObjectId	ref: Company	Reference to company if ownerType = 'company'
postedBy	ObjectId	ref: User	Actual user who created the listing
createdAt	Date	auto	Creation timestamp
updatedAt	Date	auto	Last update timestamp

Table 12: Companies Collection Structure

Field	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY, auto-generated	Unique company identifier
companyName	String	REQUIRED	Registered company name
businessRegistrationNumber	String	REQUIRED, UNIQUE	Business registration number
businessRegistrationDoc	String	REQUIRED	Cloudinary URL to registration document
industry	String	REQUIRED	Industry/sector
contactEmail	String	REQUIRED	Company contact email
contactPhone	String	REQUIRED	Company contact phone
website	String		Company website URL
address	String		Company address
foundedDate	Date		Company founding date

Field	Data Type	Constraints	Description
logo	String		Cloudinary URL to company logo
coverPhoto	String		Cloudinary URL to cover photo
description	String		Company description
socialLinks	Object		Facebook, Twitter, Instagram, LinkedIn URLs
adminUser	ObjectId	REQUIRED, ref: User	Company admin user reference
verificationStatus	String (enum)	DEFAULT 'pending': pending, verified, rejected	Verification status
rejectionReason	String		Admin rejection reason
verifiedAt	Date		Verification timestamp
verifiedBy	ObjectId	ref: User	Admin who verified
isActive	Boolean	DEFAULT true	Company active status
deletedAt	Date		Soft-delete timestamp
createdAt	Date	auto	Registration timestamp
updatedAt	Date	auto	Last update timestamp

Table 13: Categories Collection Structure

Field	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY, auto-generated	Unique category identifier
name.en	String	REQUIRED	Category name in English
name.am	String	REQUIRED	Category name in Amharic
slug	String	REQUIRED, UNIQUE	URL-friendly identifier
requiredFields	[Object]		Array of field definitions (key, label, type, options, required)
isActive	Boolean	DEFAULT true	Category active status
createdAt	Date	auto	Creation timestamp
updatedAt	Date	auto	Last update timestamp

Table 14: Conversations Collection Structure

Field	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY, auto-generated	Unique conversation identifier
listing	ObjectId	REQUIRED, ref: Listing	Listing the conversation is about
participants	[ObjectId]	INDEX, ref: User	Array of participant user IDs
lastMessage	ObjectId	ref: Message	Reference to most recent message
createdAt	Date	auto	Conversation creation timestamp
updatedAt	Date	auto	Last update timestamp

Table 15: Messages Collection Structure

Field	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY, auto-generated	Unique message identifier
conversation	ObjectId	REQUIRED, INDEX, ref: Conversation	Parent conversation reference
sender	ObjectId	REQUIRED, ref: User	Sender user reference
body	String		Message text content
attachment	String		Cloudinary URL to attached file
isRead	Boolean	DEFAULT false	Read/unread status
createdAt	Date	auto (INDEX: conversation+createdAt)	Message timestamp

Table 16: PremiumSubscriptions Collection Structure

Field	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY, auto-generated	Unique subscription identifier
user	ObjectId	REQUIRED, INDEX, ref: User	Subscribing user reference
categorySlug	String	REQUIRED, INDEX	Category being subscribed to
duration	String (enum)	REQUIRED: monthly, yearly	Subscription duration
paymentMethod	String (enum)	REQUIRED: cbe, telebirr, mpesa, chapa	Payment method
receiptUrl	String	DEFAULT ''	Cloudinary URL to uploaded receipt
fee	Number	REQUIRED	Subscription fee in ETB

Field	Data Type	Constraints	Description
status	String (enum)	DEFAULT 'pending': pending, active, inactive, expired	Subscription status
startDate	Date		Subscription start date
expiryDate	Date	INDEX	Subscription expiry date
activatedBy	ObjectId	ref: User	Admin who activated
activatedAt	Date		Activation timestamp
deactivatedBy	ObjectId	ref: User	Admin who deactivated
rejectedBy	ObjectId	ref: User	Admin who rejected
rejectionReason	String		Rejection reason
expiryNotified	Boolean	DEFAULT false	Whether expiry notification was sent
reminderNotified	Boolean	DEFAULT false	Whether 3-day reminder was sent
txRef	String		Chapa transaction reference
createdAt	Date	auto	Creation timestamp

Table 17: Notifications Collection Structure

Field	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY, auto-generated	Unique notification identifier
recipient	ObjectId	REQUIRED, INDEX, ref: User	Target user
type	String	REQUIRED	Notification type (e.g. new_message, verification_pending, subscription_activated)
title	String		Notification title
body	String		Notification body text
isRead	Boolean	DEFAULT false, INDEX	Read/unread status
relatedId	ObjectId		Related document ID
relatedModel	String		Related document model name
createdAt	Date	auto (INDEX: recipient+isRead+createdAt)	Creation timestamp

Table 18: SavedSearches Collection Structure

Field	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY, auto-generated	Unique saved search identifier
user	ObjectId	REQUIRED, ref: User	Owning user reference
params.q	String		Search keyword
params.category	String		Category slug filter
params.minPrice	Number		Minimum price filter
params.maxPrice	Number		Maximum price filter
params.region	String		Region filter
params.lat	Number		Latitude for geosearch
params.lng	Number		Longitude for geosearch
params.radius	Number		Radius in km for geosearch
createdAt	Date	auto	Save timestamp

Table 19: CompanyMembers Collection Structure

Field	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY, auto-generated	Unique member record identifier
company	ObjectId	REQUIRED, INDEX, ref: Company	Company reference
user	ObjectId	REQUIRED, INDEX, ref: User	Member user reference
memberRole	String (enum)	DEFAULT 'viewer': editor, viewer	Role within the company
status	String (enum)	DEFAULT 'pending': pending, active, declined, removed	Invitation lifecycle status
invitedBy	ObjectId	ref: User	User who sent the invitation
invitedAt	Date	DEFAULT now	Invitation timestamp
respondedAt	Date		Response timestamp
createdAt	Date	auto	Record creation timestamp
updatedAt	Date	auto	Last update timestamp

Table 20: ActivityLogs Collection Structure

Field	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY, auto-generated	Unique log entry identifier
user	ObjectId	REQUIRED, ref: User	User who performed the action
event	String	REQUIRED	Event type (e.g. login, listing_created)
metadata	Mixed		Additional event context data
createdAt	Date	auto, TTL INDEX (90 days)	Timestamp; records auto-expire after 90 days

Table 21: ModerationReports Collection Structure

Field	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY, auto-generated	Unique report identifier
reportedBy	ObjectId	REQUIRED, ref: User	User who submitted the report
targetType	String (enum)	REQUIRED: listing, message	Type of reported content
targetId	ObjectId	REQUIRED, INDEX	ID of reported document
reason	String	REQUIRED	Reason for report
status	String (enum)	DEFAULT 'open': open, actioned, dismissed	Report resolution status
actionType	String (enum)	warn_owner, remove_listing, suspend_owner, dismiss	Admin action taken
adminNote	String		Optional admin note
actionBy	ObjectId	ref: User	Admin who resolved the report
actionAt	Date		Resolution timestamp
createdAt	Date	auto (INDEX: status+createdAt)	Report creation timestamp
updatedAt	Date	auto	Last update timestamp

3.3 User Interface Design

Figure 17: Login Interface Design

Figure 18: User Dashboard Interface (Buyer/Seller)

Figure 19: Product Listing Interface

3.4 Deployment Diagram

Figure 20: Deployment Diagram

REFERENCES

Bennett, S., McRobb, S., & Farmer, R. (2010). *Object-Oriented Systems Analysis and Design Using UML* (4th ed.). McGraw-Hill.

Kendall, K. E., & Kendall, J. E. (2022). *Systems Analysis and Design* (10th ed.). Pearson.

Pressman, R. S. (2015). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill.

Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.

Ethiopian Investment Commission. (2023). *Digital Economy Statistics Ethiopia*.

Ministry of Innovation and Technology, Ethiopia. (2023). *National Digital Economy Strategy — Digital Ethiopia 2025*.

Chapa. (2024). *Chapa Payment Gateway API Documentation*. https://developer.chapa.co

AfroMessage. (2024). *AfroMessage SMS Gateway API Documentation*. https://afromessage.com

MongoDB, Inc. (2024). *MongoDB Documentation*. https://www.mongodb.com/docs

Socket.IO. (2024). *Socket.IO Documentation*. https://socket.io/docs